PROJECT STATUS REPORT

BIPE Academic Portal V2

Detailed implementation report covering the current completed scope of the new standalone BIPE portal built with PHP, MySQL, HTML, CSS, and JavaScript.

ADMIN PAGES	FACULTY PAGES	STUDENT PAGES	DATABASE TABLES
14	11	7	19

Prepared On

23 March 2026

Workspace

`D:\Projects\BIPE Portal`

Project Status

Standalone portal implemented and production-hardened at application level.

Report Scope

Everything completed in the project up to the current state, including features, architecture, database, security, audit logging, error handling, and operational readiness.

1. Executive Summary

The BIPE Academic Portal V2 is a complete standalone replacement for the earlier single-file BIPE portal concept. The current system is implemented as a multi-page PHP application backed by a MySQL database and a dedicated front-end asset structure. The portal now supports separate administrator, faculty, and student workflows; persistent database-backed records; approval flows; attendance management; marks management; assignment tracking; report cards; audit logging; backup and restore; password reset flows; IP-based activity logging; and production-oriented safeguards.

The project no longer depends on `BIPE_Portal.html` or `BIPE_Combined_Portal.html` at runtime. Those files act only as reference sources for feature parity and design guidance. The live portal uses its own PHP pages, CSS files, JavaScript files, database schema, seeds, and supporting infrastructure.

> **Current completion position:** the application is functionally complete for the implemented academic workflows and has been upgraded with production-style safeguards such as CSRF protection, session hardening, rate limiting, password policy enforcement, audit logging with IP capture, styled confirmation modal handling, and a dedicated error handling page.

2. Project Goals and Transformation Scope

Original Requirement Direction

- Build a completely new website, not a runtime wrapper around the legacy HTML file.
- Retain the features and design behavior from the original BIPE portal references.
- Use PHP and MySQL for real data persistence.
- Organize each page with dedicated CSS and JS assets.
- Ensure responsiveness and professional styling.

Expanded Implemented Scope

- Admin, faculty, and student role separation.
- Database schema, seed data, and migration support.
- Security hardening and production-readiness improvements.
- Audit log with user IP visibility in admin panel.
- Dedicated error handling page and styled confirmation modal.

3. Technology Stack

LAYER	TECHNOLOGY	CURRENT USE
Server	PHP	Page rendering, authentication, DB access, business logic, security handling, error handling.
Database	MySQL via PDO	Persistent storage for admins, teachers, students, subjects, marks, attendance, assignments, OTPs, audit logs, and backups.
Frontend	HTML, CSS, JavaScript	Responsive multi-page UI with dedicated page-level asset files.
Asset Model	Per-page CSS and JS	Each major screen maps to separate style and behavior files under `assets/css` and `assets/js`.
Runtime Support	Storage directories	Application logs and rate-limit cache files in `storage/logs` and `storage/cache`.

4. Current Project Structure

AREA	PURPOSE
`admin/`	Administrator workflows such as students, faculty, attendance oversight, marks management, report cards, audit logs, backup/restore, reports, and settings.
`faculty/`	Faculty workflows for attendance, records, marks upload, marks view, assignments, reports, dashboard, profile, login, register, and forgot password.
`student/`	Student workflows for dashboard, attendance, marks, assignments, login, register, and forgot password.
`app/`	Bootstrap, helpers, DB connection, authentication, actions, layouts, error rendering, and portal feature services.
`database/`	Schema, seed files, and migration scripts.
`assets/`	Shared and page-specific CSS/JS assets plus images.
`storage/`	Logs and runtime cache files.

File distribution summary: 14 admin PHP pages, 11 faculty PHP pages, 7 student PHP pages, 35 CSS files, and 35 JS files are currently present in the project structure.

5. Implemented Role-Based Functional Modules

Administrator Portal

dashboard.php | students.php | faculty.php | attendance.php | marks.php | assignments.php | reports.php | report_cards.php | report_card_print.php | audit_log.php | backup.php | settings.php | login.php | forgot.php

- Manage student master data, manual entry, CSV bulk import, search, filter, and deletion.
- Approve, reject, edit, delete, and review faculty records.
- Review attendance history and holiday/event records.
- Manage marks overview, lock sections by department and semester, and export summaries.
- View assignment tracking summaries.
- Generate class report cards and printable report card views.
- Inspect audit logs with search and now IP visibility.
- Export and restore backup data.
- Change password, manage academic year reset logic, mark types, and portal-wide settings.

Faculty Portal

dashboard.php | attendance.php | records.php | marks.php | view_marks.php | assignments.php | reports.php | profile.php | login.php | register.php | forgot.php

- Mark class attendance and edit previously saved attendance records.
- Upload marks manually or by CSV, including absent status handling.
- Review saved marks uploads by subject and mark type.
- Track assignment submission status student-wise.
- View department academic summaries and recent activity.
- Maintain faculty profile including password change.
- Register for admin approval and use OTP-based password reset.

Student Portal

dashboard.php | attendance.php | marks.php | assignments.php | login.php | register.php | forgot.php

- Register against roster data already present in the database.
- Login and view attendance, marks, assignments, and profile/dashboard details.

- Use OTP-based password reset on registered email.
- See absent marks as AB where applicable.

6. Database Design and Data Model

The schema currently defines 19 core tables and one migration file for incremental upgrade. The system is normalized enough to support separate entities, transactional updates, and cross-role academic workflows.

TABLE	ROLE IN SYSTEM
`academic_years`	Stores current and past academic year windows.
`admins`	Administrator credentials and profile data.
`departments`	Department master data and theme identity.
`teachers`	Faculty account data, approval state, and department mapping.
`students`	Student roster, registration state, email, and semester mapping.
`subjects`	Subject offerings by department and semester.
`mark_types`	Configurable types such as Sessional, Pre Board, Assignment, Attendance, Practical.
`mark_locks`	Stores lock state by department and semester.
`admin_otp_requests`	Admin password reset OTP requests.
`password_reset_otps`	Teacher/student reset OTP storage.
`holiday_events`	Holiday and event records impacting attendance workflows.
`attendance_sessions`	High-level attendance sheet/session entries.
`attendance_records`	Per-student attendance rows.
`mark_uploads`	Marks upload headers by subject, semester, and mark type label.
`mark_records`	Per-student marks rows, including absent support.
`assignments`	Assignment tracker header rows.
`assignment_submissions`	Per-student submission status rows.
`audit_logs`	Role-based audit trail now including stored IP address.

> **Seed and migration support:** `seed_core.sql` provides admin, academic year, departments, subjects, and default mark types. `seed_students.sql` contains the

> student roster. `migration_add_audit_ip.sql` upgrades an existing audit table with IP logging support.

7. Key Functional Enhancements Already Completed

Combined Portal Feature Integration

- Marks management
- Report cards
- Audit logs
- Backup and restore
- Faculty dashboard
- Faculty marks viewing
- Faculty profile
- Student and faculty OTP reset

Design and Branding Completion

- Logo integrated from uploaded image
- Index page background updated with provided image
- Footer added with college contact/social details
- Consistent page-level CSS/JS structure maintained
- Responsive layouts preserved across modules

8. Security and Production Hardening Status

AREA	IMPLEMENTED STATE	IMPACT
Sessions	Hardened cookies, strict mode, idle timeout, logout session rotation	Improves account session safety.
CSRF	POST request token protection with automatic token injection	Protects against cross-site form submission.
Rate Limiting	Server-side rate limiting for login and OTP actions	Reduces brute-force and repeated reset abuse.
Password Policy	Strong password validation on registration, reset, and update	Improves credential quality.
Headers	CSP, X-Frame-Options, no-sniff, referrer policy, permissions policy	Hardens client-side delivery and framing/security posture.
Error Handling	Dedicated error page with code, type, and message	Improves diagnosability and user-facing professionalism.
Uploads	Validated CSV and JSON upload handling	Reduces malformed or oversized file risk.
Audit Logging	Role-aware logs plus IP capture and admin panel visibility	Improves traceability and reviewability.

9. Audit, Monitoring, and Error Handling

Audit Logging

- Administrative, faculty, and student actions are recorded through a centralized audit helper.
- Audit entries include role, user identifier, action code, details, timestamp, and IP address.
- The administrator panel displays audit rows with search support for user, IP, action, and details.

Error Handling Page

- A dedicated error renderer now produces a styled error page rather than raw fallback markup.
- The error page shows error code, error type, and the message describing what happened.
- Unhandled exceptions and CSRF validation failures can render through the error handling layer.

Logging Support

- Runtime application logging is supported through `storage/logs/app.log`.
- Rate-limit support uses cache files under `storage/cache`.

10. UX and Interaction Improvements

- Responsive dashboard and auth layouts remain in place.
- Flashes auto-dismiss for a cleaner operator experience.
- File input helpers show selected file names in UI.
- Confirmation interactions now use a styled site modal instead of raw browser confirm prompts.
- Logout now asks for confirmation with explicit logout and cancel choices.

11. Deployment and Configuration Readiness

Ready in Code

- Environment-driven configuration template present
- Security flags available
- Mail delivery switch available
- Migration support available
- Database import order documented

Still Required on Server

- Real production DB credentials
- HTTPS deployment and secure cookie usage
- Working mail delivery for OTP reset in production
- Write permissions for `storage/logs` and `storage/cache`
- `pdo_mysql` enabled in PHP

12. Verification and Validation Status

PHP syntax verification has been run across the full project and all PHP files currently pass `php -l`.

The project has been repeatedly updated and revalidated after feature additions such as production hardening, IP logging, the styled error page, and the confirmation modal.

> **Important realism note:** code-level verification is complete at the syntax level, but a live end-to-end production deployment still depends on server configuration, mail delivery, MySQL import, HTTPS, and environment variable setup.

13. Current Completion Status

AREA	STATUS	REMARKS
Standalone portal architecture	**Completed**	No runtime dependency on old BIPE HTML files.
Multi-role portals	**Completed**	Admin, faculty, and student modules implemented.
Database schema and seeds	**Completed**	Schema, seed files, and audit IP migration exist.
Combined portal feature parity	**Substantially completed**	All major functional modules from the combined reference were integrated into the PHP system.
Production hardening	**Completed at application level**	Security controls and operational safeguards implemented in code.
Error handling page	**Completed**	Styled page shows code, type, and message.
Audit logging with IP	**Completed**	Stored in DB and shown in admin panel.
Deployment on live server	**Pending environment work**	Needs real server credentials, HTTPS, and mail delivery setup.

14. Suggested Next Optional Enhancements

- Custom 404 handling for missing routes.
- More granular audit filters such as dedicated IP filter and date range filter.
- Server-integrated SMTP mail delivery instead of plain PHP mail fallback.
- Automated deployment guide for Apache/XAMPP, shared hosting, or VPS.
- Functional browser-based smoke tests after full server deployment.

This report captures the project as completed up to the current state in the workspace on 23 March 2026.